

Then simple addition yields the above-mentioned dividend in the form

$$a_1b_2 + 2^{-47}(a_1b_3 + a_2b_2): \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 0 & g & g & \dots & \dots & g & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & g & g & \dots & \dots & g & & \\ \hline \end{array} \quad (P15)$$

where g signifies any significant bit and the leftmost g in the left cell could be 1. Note that the inequalities (12) show

$$a_1b_2 + 2^{-47}(a_1b_3 + a_2b_2) < 2^{-1}.$$

Now we divide this result by b_1 . Since we must have a double-length quotient and since each fixed-point division which we are to use now gives a single-length quotient and a single-length remainder, we first divide the double-length dividend by the single-length divisor b_1 , and obtain the single-length quotient q_1 and a single-length remainder. We then augment this remainder with a single-length zero (thus obtaining the double-length number) and divide this double-length number by b_1 to obtain the single-length quotient q_2 . The q_1 and q_2 constitute the desired double-length quotient $(a_1b_2 + 2^{-47}(a_1b_3 + a_2b_2))/b_1$. Thus we now have V in the form

$$V: \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 47 \text{ zeros} & & & & & & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & g & g & \dots & \dots & g & & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & g & g & \dots & \dots & g & & \\ \hline \end{array} \quad (P16)$$

Step 3. Compute $U - V$.

To compute $U - V$ by adding the complemented V to U , namely, $U - V = U + (-V)$, we proceed as follows. First, subtract 1 from the rightmost bit of the leftmost word of U (borrow) and add 1 to the rightmost bit of the rightmost word of U (end-around carry). Call the result U' .

Next, complement each bit g of V in (P16). Call the result V' .

$$V': \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 47 \text{ zeros} & & & & & & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & \bar{g} & \bar{g} & \dots & \dots & \bar{g} & & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & \bar{g} & \bar{g} & \dots & \dots & \bar{g} & & \\ \hline \end{array} \quad (P17)$$

where $\bar{g}$ means the complemented g . Add U' ((P14)) and V' bit-by-bit using fixed-point operations, where any carry bit that might be produced from the addition of the lower two words must be properly added to the next upper word. We again arrange the result ($= U - V$) in the form

$$U - V: \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 0 & f & f & \dots & \dots & f & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & f & f & \dots & \dots & f & & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & f & f & \dots & \dots & f & & \\ \hline \end{array} \quad (P18)$$

Step 4. Compute $U - V/b_1 = \text{Ans.}$

We must now divide the triple-length number $U - V$ by the single-length number b_1 to obtain the triple-length quotient. We accomplish this by three successive fixed-point divisions, each time obtaining a single-length quotient and a single-length remainder. Hence it is vital to obtain the correct remainder at least

from the first two divisions. To obtain this we regard the dividend $U - V$ and the divisor b_1 as integers and use the integer divide operation. (The fractional divide operation may not retain the last bit of the remainder.) It is then necessary to rearrange $U - V$ of (P18) in the form

$$U - V: \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & f & f & \dots & \dots & f \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline f & f & \dots & \dots & f & & & \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|c|c|} \hline f & f & \dots & \dots & f & 0 & & \\ \hline \end{array} \quad (P19)$$

↑
an extra zero

(We do not have to rearrange b_1 .) Now the proposed division $(U - V)/b_1$ can be easily performed by three successive single-precision fixed-point divisions, thus obtaining the triple-length quotient. The interpretation of the result and the adjustment of the binary point are easy.

4. Remarks

Four triple-precision (132-bit) floating-point arithmetic subroutines (addition, subtraction, multiplication and division subroutines) have actually been coded for the Control Data 1604 computer using the described algorithm and they are now working at the University of Texas Computation Center. The Control Data 3600 computer version of the above subroutines are also working at Argonne National Laboratory.

Acknowledgments. This work was supported in part by the National Science Foundation under Grant GP-217 and by the U.S. Army Research Office (Durham) under Grant DA-ARO(D)-31-124-G388. The technique explained in this note was first developed at the Applied Mathematics Division, Argonne National Laboratory, Argonne, Illinois, during the summer of 1963 while I was working there as a Resident Student Associate. Many thanks are due to Dr. Robert T. Gregory of the University of Texas for reading the rough draft of this note. I am very much indebted to Dr. Richard F. King of Argonne National Laboratory for his many constructive suggestions.

RECEIVED SEPTEMBER, 1964

REFERENCE

1. GREGORY, R. T., AND RANNEY, J. L. Floating-point arithmetic with 84-bit numbers. *Comm. ACM* 7, 1 (Jan. 1964), 10-13.

Method in Randomness

MARTIN GREENBERGER

Massachusetts Institute of Technology, Cambridge, Mass.

Certain nonrandom properties of a commonly used random number generator are described and analyzed.

Introduction

Almost all prescribers of random number generators label their concoctions with a virtual skull and crossbones that warns the user against indiscriminate application. The label usually consists of a qualifier for random, like *pseudo* or *quasi*, plus a few general words of caution.

There is more to this than a simple hedging. Most generators in use are of the congruential type, wherein each number of the generated series is calculated solely from its predecessor and some fixed parameters. A typical generator, for example, has the form

$$x_{i+1} \equiv \lambda x_i + c \pmod{2^p} \quad (1)$$

where x_{i+1} is the number being generated, x_i is its predecessor, λ and c are fixed parameters of the generator, and 2^p is the modulus.

On first glance, it is not clear how a simple rule like (1) can produce numbers that are random even in pretense; that is, pseudo-random. Obviously, it cannot produce true randomness. There is blatant linkage between x_i and x_{i+1} .

A closer look reveals that reduction modulo $2''$ performs a scrambling, not unlike successive spins of a roulette wheel. How complete is the scrambling? That depends on the choice of parameters.

There has been some number-theoretic work recently relating the values of parameters to certain statistical properties of the resultant generator [1-6]. This work has not been sufficiently comprehensive, however, to indicate a choice of parameters that insures against all of the unlimited possibilities for nonrandomness. The difficulties are philosophical as well as deductive.

The result is that parameters still are selected primarily on the basis of a limited set of *desiderata* for the generator, such as high speed, long period, and negligible first-order serial correlation. This is why a warning is in order. The user is advised to custom design his own statistical tests to make sure that the generator is acceptable for his particular application.

Nonrandomness

One random number generator which behaved non-randomly was the midsquare method, proposed by John von Neumann in the 1940's. As von Neumann subsequently pointed out, the method had a propensity for producing nothing but zeros after a certain point. In response to attempts to modify it, von Neumann remarked that "ingenious tricks to overcome the zero mechanism might just bring out the next most disastrous mechanism" [9].

A somewhat more subtle kind of nonrandom behavior, this time in the congruential method, was recently called to the author's attention by George Sadowsky of the Yale Computer Center. Seven years ago the author examined several classes of random number generators analytically, and applied many of the standard statistical tests [8]. One of the generators was

$$x_{i+1} \equiv (2^{18} + 3)x_i \pmod{2^{35}}. \quad (2)$$

A multiplier of $(2^{18} + 3)$ allowed one shift and three add operations to replace the multiply operation. This provided a distinct speed advantage on the IBM 704 computer, while still providing a period of over 8 billion and negligible first-order serial correlation.

On the basis of these credentials, a number of computer installations adopted this generator for the library tapes of their operating systems. Yale University was one of them. This past year, Joseph Lach used the generator at Yale (called RANNO there) during the course of his Ph.D. research on a problem in physics. His suspicions about second-order correlation led him to the following experiment [7].

Lach generated successive members of the series until reaching a number less than .1; that is, an $x_i < 2^{35}/10$. He would then plot a point on the oscilloscope, taking the next two members of the series as coordinates. Figure 1 shows the results of his plotting 50,000 points, while Figure 2 gives the corresponding results of another generator for comparison. The black bands in Figure 1 are devoid of points.

The author re-examined RANNO after being advised of Lach's finding and discovered that the problem was even more serious than Figure 1 suggests. This can be seen from the following simple analysis:

$$x_{i+1} \equiv (2^{18} + 3)x_i \pmod{2^{35}}, \quad (2)$$

$$\begin{aligned} x_{i+2} &\equiv (2^{18} + 3)^2 x_i \\ &\equiv [6(2^{18} + 3) - 9]x_i \pmod{2^{35}}, \\ &\equiv 6x_{i+1} - 9x_i \end{aligned} \quad (3)$$

For convenience, we define $\mu_j = 2^{-35}x_j$. Then $0 \leq \mu_j \leq 1$ for $j = i + 1, i + 2$, and we can view Figures 1 and 2 as unit squares. Furthermore, the congruence

$$x \equiv 6x_{i+1} \pmod{2^{35}} \quad (4)$$

maps into the six equalities

$$\begin{aligned} \mu &= 6 \left(\mu_{i+1} - \frac{k}{6} \right) \\ \frac{k}{6} &\leq \mu_{i+1} \leq (k + 1)/6 \quad (k = 0, 1, 2, \dots, 5) \end{aligned} \quad (5)$$

which define the right sides of the six black bands of Figure 1.

As μ_i ranges over the restrictive subinterval $(0, .1)$, $9\mu_i$ is confined to $(0, .9)$, whereas μ_{i+1} covers the complete unit interval. By congruence (3), μ_{i+2} is excluded from the intervals that rise a vertical distance of .1 above each μ of (5). The resulting voids are the six black bands of Figure 1. We can join them into one band by rolling Figure 1 into a horizontal cylinder. Call .1 the *width* of this band.

Bands of other widths can be obtained by placing different restrictions on μ_i . In general, black bands of width equal to $2t$ ($0 < t < 1/2$) are produced by restricting μ_i to the union of the 9 subintervals

$$\begin{aligned} (n - 1 + t)/9 \leq \mu_i \leq (n - t)/9 \\ (n = 1, 2, \dots, 9). \end{aligned} \quad (6)$$

Conversely, white bands of the same width result from the complementary set of restrictions

$$\begin{aligned} 0 \leq \mu_i \leq t/9, \\ (n - t)/9 \leq \mu_i \leq (n + t)/9 \quad (n = 1, 2, \dots, 8), \\ (1 - t/9) \leq \mu_i \leq 1. \end{aligned} \quad (7)$$

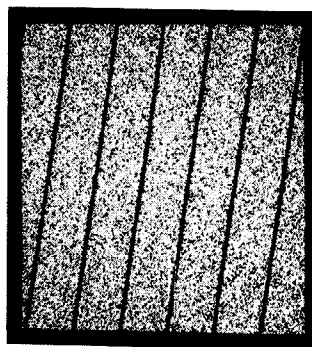


FIG. 1

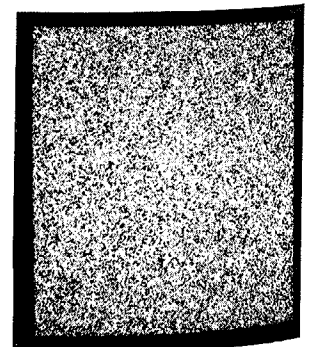


FIG. 2

The line around the cylinder defined by (5) bisects the band in either case.

Conclusion

The aberration in this generator was obvious once we knew where to look. In a generator whose parameters are less transparent, it may be much more difficult to know where to look, and the aberration may be much more complex in nature; but there is no a priori reason to suppose that a similar aberration does not exist.

The corrective procedure that Lach adopted was to switch back and forth among five variations of the generator, using the following five multipliers: 186277, 186285, 186293, 186301 and 186309. Each of these multipliers approximates $\sqrt{2^{35}}$ and each of them is congruent to ± 3 modulo 8, but none of them equals $(2^{18} + 3)$.

Switching among some such set of generators may intuitively seem to resolve the problem, particularly if the switching is done "at random." Recalling von Neumann's observation, however, the user would be well advised to heed the customary warning and construct his own tests.

RECEIVED AUGUST, 1964

REFERENCES

1. COVEYOU, R. R. Serial correlation in the generation of pseudo-random numbers. *J. ACM* 7 (1960), 72-74.
2. FRANKLIN, J. N. Deterministic simulation of random processes. *Math. Comput.* 17 (Jan. 1963), 28.
3. GREENBERGER, M. Notes on a new pseudo-random number generator. *J. ACM* 8 (1961), 163-167.
4. —. An a priori determination of serial correlation in computer generated random numbers. *Math. Comput.* 15 (1961), 383-389.
5. JAGERMAN, D. The autocorrelation and joint distribution functions of the sequences $\left\{ \frac{a}{m} j^2 \right\}$, $\left\{ \frac{a}{m} (j+r)^2 \right\}$. *Math. Comput.* 18 (Apr. 1964), 211.
6. JANSSON, B. Autocorrelations between pseudo-random numbers. *BIT* 4, 1 (1964), 6-27.
7. LACH, J. Random number generators. Yale Computer Ctr. Mem. No. 27, Oct. 1963.
8. ORCUTT, G. H., GREENBERGER, M., KORBEL, J., AND RIVLIN, A. M. Appendix to Part IV. In *Microanalysis of Socio-economic Systems*, Harpers, 1961.
9. VON NEUMANN, J. *Various Techniques Used in Connection with Random Digits*. N.B.S. Appl. Math. Series No. 12, U. S. Government Printing Off., Washington, D. C., 1951, 36-38.

On a Divide-and-Correct Method For Variable Precision Division

E. V. KRISHNAMURTHY

*The University of Chicago, * Illinois*

Described in this paper is a divide-and-correct method for variable precision division in digital computers. Unlike the earlier methods of Stein and Pope, the present method uses a suitably rounded form of the normalized divisor for getting an estimate of the quotient characters. This results in a correction of at most plus or minus one to the estimate, to obtain the exact quotient character. It is believed that this method will be widely applicable for division operations in variable word-length character-oriented machines.

Introduction

The problem of variable precision division arises when a computer has to handle both a variable precision divisor and a dividend, while its arithmetic unit is only capable of performing the division of a double precision (two character) dividend by a single precision (single character) divisor. Usually, it is required that the resulting quotient occupies the same storage registers as the original dividend at the termination of the division operation (single address operation). In such a case, the most convenient way to

solve the problem is to use a divide-and-correct method. The methods described in papers by Stein and Pope [1] and Stein [2] obtain an estimate of the quotient characters by dividing the two leading characters of the current partial remainder either by the leading character of the normalized divisor or by the leading character of the normalized divisor increased by unity. The present method uses a rounded form of the normalized divisor for obtaining an estimate. This is advantageous due to the fact that the correction to the estimate is at most plus or minus one; this correction is governed by certain simple logical rules which depend upon the sign of the roundoff factor and the sign of the remainder obtained in the two character by one character division process. In a sense, this method is an extension of the recent method of segmented division proposed by Metropolis and Ashenhurst [3].

The Division Method

Definition and Preliminary Operations. Let us assume that the dividend a , the divisor b and the quotient q are written as

$$a = \sum_{j=0}^n a_j \beta^j \cdot e_a \quad (1)$$

$$b = \sum_{j=0}^d b_j \beta^j \cdot e_b \quad (2)$$

$$q = \sum_{j=0}^n q_j \beta^j \cdot e_q \quad (3)$$

* Institute for Computer Research.